Modül 13

Directives

Derinlemesine — Attribute, Structural, Composition

Modül:	13 / 30
Ders Sayısı:	7 ders
Seviye:	İleri
Versiyon:	Angular v21

Hazırlayan: Claude • Türkçe Angular v21 Eğitimi

Modül 13'e Hoş Geldin

Modül 6'da temellerini gördüğümüz directive'leri bu modülde profesyonel seviyede inceliyoruz. Production-grade directive'ler, advanced pattern'ler, ve directive composition.

Öğrenecekler:

- ✓ Attribute directive'leri tüm detaylarıyla
- ✓ Structural directive yazmak (*directive syntax)
- ✓ Host bindings ile component davranışı
- ✓ Directive composition (v15+)
- ✓ hostDirectives ile attribute reuse
- ✓ NgComponentOutlet ile dinamik component
- ✓ NgTemplateOutlet ile dinamik template

DERS 13.1

Attribute Directive — Production Patterns

Basit directive'lerin ötesinde — production örnekleri.

🔒 ClickOutside Directive

```
@Directive({
  selector: '[clickOutside]',
  host: { '(document:click)': 'onClick($event)' }
})
export class ClickOutsideDirective {
  clickOutside = output<void>();
  private el = inject(ElementRef);

  onClick(event: MouseEvent) {
    const target = event.target as Node;
    if (!this.el.nativeElement.contains(target)) {
      this.clickOutside.emit();
    }
  }
}

// Kullanım — Dropdown kapatma
@Component({
  template: `
    <div clickOutside (clickOutside)="isOpen.set(false)">
      <button (click)="isOpen.update(v => !v)">Menu</button>
      @if (isOpen()) {
        <ul>...</ul>
      }
    </div>
  `
})
```

🔒 Debounce Click Directive

```
@Directive({
  selector: '[debounceClick]',
  host: { '(click)': 'onClick($event)' }
})
export class DebounceClickDirective {
  delay = input(500);
  debounceClick = output<MouseEvent>();
  private lastClick = 0;

  onClick(event: MouseEvent) {
    const now = Date.now();
    if (now - this.lastClick >= this.delay()) {
      this.lastClick = now;
      this.debounceClick.emit(event);
    }
  }
}

// Kullanım
// <button debounceClick [delay]="1000" (debounceClick)="save()">Kaydet</button>
```

□ Lazy Image Directive (Intersection Observer)

```
@Directive({
  selector: 'img[lazyLoad]'
})
export class LazyLoadDirective implements OnInit, OnDestroy {
  lazyLoad = input.required<string>();  // Real src
  private el = inject(ElementRef<HTMLImageElement>);
  private observer?: IntersectionObserver;

  ngOnInit() {
    this.observer = new IntersectionObserver((entries) => {
      entries.forEach(entry => {
        if (entry.isIntersecting) {
          this.el.nativeElement.src = this.lazyLoad();
          this.observer?.disconnect();
        }
      });
    });
    this.observer.observe(this.el.nativeElement);
  }

  ngOnDestroy() {
    this.observer?.disconnect();
  }
}

// <img lazyLoad="/photo.jpg" alt="..." />
```

DERS 13.2

Structural Directive Yazmak

*ngIf benzeri kendi structural directive'ini yarat.

Structural Directive Nedir?

Element'i DOM'a ekleyip çıkartan directive'ler. * ile başlar. Aslında syntactic sugar — arka planda <ng-template>.

Basit Örnek: *appUnless

```
import { Directive, Input, TemplateRef, ViewContainerRef, inject } from
'@angular/core';

@Directive({
  selector: '[appUnless]',
})
export class UnlessDirective {
  private templateRef = inject(TemplateRef<unknown>);
  private viewContainer = inject(ViewContainerRef);
  private hasView = false;

  @Input() set appUnless(condition: boolean) {
    if (!condition && !this.hasView) {
      this.viewContainer.createEmbeddedView(this.templateRef);
      this.hasView = true;
    } else if (condition && this.hasView) {
      this.viewContainer.clear();
      this.hasView = false;
    }
  }
}

// Kullanım — ngIf'in tersi
// <p *appUnless="isLoggedIn">Lütfen giriş yapın</p>
```

Context Passing — *appFor

```
@Directive({
  selector: '[appFor]'
})
export class ForDirective<T> {
  private templateRef = inject(TemplateRef<{ $implicit: T; index: number }>);
  private viewContainer = inject(ViewContainerRef);

  @Input() set appForOf(items: T[]) {
    this.viewContainer.clear();
    items.forEach((item, index) => {
      this.viewContainer.createEmbeddedView(this.templateRef, {
        $implicit: item,   // Default context value
        index
      });
    });
  }
}

// Kullanım
// <li *appFor="let item of items; let i = index">
//   {{ i }}. {{ item }}
// </li>
```

⚠ Modern Angular Uyarısı

🤔 Hala Gerekli mi?

Modern Angular'da `@if`, `@for`, `@switch` built-in olduğu için çoğu structural directive ihtiyacı ortadan kalktı. Custom structural directive yazmak nadir bir durum oldu.

DERS 13.3

Host Bindings — Modern Pattern

Component/directive host element'ine class, style, attribute, event ekleme.

host Property

```
@Component({
  selector: 'app-card',
  host: {
    // Class binding
    'class': 'card',                    // Static
    '[class.active]': 'isActive()',     // Dynamic
    '[class.disabled]': 'isDisabled()',

    // Style binding
    '[style.background]': 'color()',

    // Attribute
    'role': 'article',                  // Static
    '[attr.aria-label]': 'label()',     // Dynamic
    '[attr.data-id]': 'id()',

    // Event listener
    '(click)': 'onClick($event)',
    '(mouseenter)': 'isHovered.set(true)',
    '(mouseleave)': 'isHovered.set(false)',
  },
  template: `<ng-content />`
})
export class CardComponent {
  isActive = input(false);
  isDisabled = input(false);
  color = input('white');
  label = input.required<string>();
  id = input.required<string>();
  isHovered = signal(false);

  onClick(e: MouseEvent) { /* ... */ }
}
```

host vs @HostBinding/@HostListener

Konu	host property	@HostBinding/@HostListener
Modernlik	✓ Modern, önerilen	Eski (hala çalışır)
Tek yerde toplu	✓ Decorator'da	Property'lerde dağınık

Performance	Aynı	Aynı
Syntax	String key	Decorator

DERS 13.4

hostDirectives — Composition (v15+)

Bir component'e başka directive'leri "bağlama" — composition pattern.

hostDirectives Nedir?

Composition over inheritance. Component'e başka directive'lerin davranışını otomatik ekle. Inherit etmeden, plug-in gibi.

Örnek: Tooltip + Highlight + Track

```
// Bağımsız directive'ler
@Directive({ selector: '[tooltip]' })
export class TooltipDirective { /* ... */ }

@Directive({ selector: '[highlight]' })
export class HighlightDirective { /* ... */ }

@Directive({ selector: '[trackClick]' })
export class TrackClickDirective { /* ... */ }

// Component — hepsini compose et
@Component({
  selector: 'app-button',
  template: `<button><ng-content /></button>`,
  hostDirectives: [
    {
      directive: TooltipDirective,
      inputs: ['tooltip: tooltipText'],  // Rename
    },
    HighlightDirective,
    {
      directive: TrackClickDirective,
      outputs: ['clicked'],
    }
  ]
})
export class ButtonComponent {
  // Otomatik:
  // - tooltipText input (TooltipDirective'den)
  // - highlight davranışı (HighlightDirective'den)
  // - clicked output (TrackClickDirective'den)
}

// Kullanım
// <app-button tooltipText="Tıkla" (clicked)="save()">Kaydet</app-button>
```

⚡ Avantajları

- ✓ Inheritance kötülüğünden kaç (TypeScript class hierarchy)
- ✓ Aynı davranışı birden fazla component'e plug-in gibi ekle
- ✓ Mixin pattern'in modern Angular versiyonu
- ✓ Test etmek daha kolay (her directive bağımsız)

DERS 13.5

NgComponentOutlet — Dinamik Component

Hangi component'i göstereceğini runtime'da belirle.

Use Case

Form alanları, plugin sistemi, dinamik widget'lar — runtime'da component'i bilmiyorsun.

Basit Kullanım

```
@Component({
  imports: [NgComponentOutlet],
  template: `
    <ng-container *ngComponentOutlet="currentComponent()" />
  `
})
export class DynamicComponent {
  currentComponent = signal<Type<any>>(HomeComponent);

  showSettings() {
    this.currentComponent.set(SettingsComponent);
  }
}
```

Input Passing

```
@Component({
  imports: [NgComponentOutlet],
  template: `
    <ng-container
      *ngComponentOutlet="
        currentComponent();
        inputs: componentInputs();
      "
    />
  `
})
export class DynamicComponent {
  currentComponent = signal<Type<any>>(UserCardComponent);

  componentInputs = computed(() => ({
    user: this.user(),
    onSave: (data: any) => this.handleSave(data),
  }));
}
```

🔌 Plugin System Example

```
interface Widget {
  name: string;
  component: Type<unknown>;
  inputs?: Record<string, unknown>;
}

@Component({
  template: `
    @for (widget of widgets(); track widget.name) {
      <div class="widget">
        <h3>{{ widget.name }}</h3>
        <ng-container *ngComponentOutlet="
          widget.component;
          inputs: widget.inputs;
        " />
      </div>
    }
  `
})
export class DashboardComponent {
  widgets = signal<Widget[]>([
    { name: 'Sales', component: SalesWidget, inputs: { period: 'month' } },
    { name: 'Users', component: UsersWidget },
    { name: 'Activity', component: ActivityWidget, inputs: { limit: 10 } },
  ]);
}
```

DERS 13.6

NgTemplateOutlet — Dinamik Template

Template fragment'ları reuse et.

Temel Kullanım

```
@Component({
  imports: [NgTemplateOutlet],
  template: `
    <!-- Template tanımla -->
    <ng-template #greeting let-name>
      <h1>Hoşgeldin, {{ name }}!</h1>
    </ng-template>

    <!-- Birden fazla yerde kullan -->
    <header>
      <ng-container *ngTemplateOutlet="greeting; context: { $implicit: 'Ali' }" />
    </header>

    <main>
      <ng-container *ngTemplateOutlet="greeting; context: { $implicit: 'Ayşe' }" />
    </main>
  `
})
```

Customizable Component Pattern

```
// Generic Card component
@Component({
  selector: 'app-card',
  imports: [NgTemplateOutlet],
  template: `
    <div class="card">
      <header>
        <ng-container *ngTemplateOutlet="headerTpl() || defaultHeader" />
      </header>
      <main>
        <ng-content />
      </main>
      <footer>
        <ng-container *ngTemplateOutlet="footerTpl() || defaultFooter" />
      </footer>
    </div>

    <ng-template #defaultHeader>
      <h2>Default Header</h2>
    </ng-template>
    <ng-template #defaultFooter>
      <p>Default Footer</p>
    </ng-template>
  `
})
export class CardComponent {
  headerTpl = input<TemplateRef<unknown> | null>(null);
  footerTpl = input<TemplateRef<unknown> | null>(null);
}

// Kullanım
@Component({
  template: `
    <ng-template #myHeader>
      <h1>Custom Header!</h1>
    </ng-template>

    <app-card [headerTpl]="myHeader">
      <p>Content here</p>
    </app-card>
  `
})
```

DERS 13.7

Production Patterns

Gerçek projelerden çıkmış directive pattern'leri.

🔐 Permission-based UI

```
@Directive({
  selector: '[appHasPermission]',
})
export class HasPermissionDirective {
  private templateRef = inject(TemplateRef<unknown>);
  private viewContainer = inject(ViewContainerRef);
  private auth = inject(AuthService);

  @Input() set appHasPermission(permission: string) {
    this.viewContainer.clear();
    if (this.auth.hasPermission(permission)) {
      this.viewContainer.createEmbeddedView(this.templateRef);
    }
  }
}

// Kullanım
// <button *appHasPermission="'admin.delete'">Sil</button>
// Sadece "admin.delete" izni olan kullanıcı görür
```

📋 Copy to Clipboard

```
@Directive({
  selector: '[copyToClipboard]',
  host: { '(click)': 'copy()' }
})
export class CopyToClipboardDirective {
  copyToClipboard = input.required<string>();
  copied = output<void>();

  async copy() {
    try {
      await navigator.clipboard.writeText(this.copyToClipboard());
      this.copied.emit();
    } catch (e) {
      console.error('Copy failed:', e);
    }
  }
}

// <button [copyToClipboard]="apiKey()" (copied)="showToast()">Kopyala</button>
```

□ Auto Focus + Select

```
@Directive({
  selector: '[autoFocus]'
})
export class AutoFocusDirective {
  shouldSelect = input(false, { alias: 'autoFocusSelect' });
  private el = inject(ElementRef<HTMLInputElement>);

  constructor() {
    afterNextRender(() => {
      this.el.nativeElement.focus();
      if (this.shouldSelect()) {
        this.el.nativeElement.select();
      }
    });
  }
}

// <input autoFocus />
// <input autoFocus autoFocusSelect />  - focus + selectAll
```

Modül 13 Özeti

Neler Öğrendin?

- ✓ Production-grade attribute directive'ler
- ✓ Custom structural directive yazma
- ✓ Host bindings modern pattern (host property)
- ✓ hostDirectives ile composition
- ✓ NgComponentOutlet (dinamik component)
- ✓ NgTemplateOutlet (dinamik template)
- ✓ Reusable directive pattern'leri

Sıradaki Modül

Modül 14 — State Management (NgRx, Signal Store). Büyük uygulamalarda state yönetimi, NgRx, Signal Store, ve state pattern'leri.